

SPORT CONNECT

Find your team, play your sport

Inspiration Lab

Presented by

Alejandro Villarreal
Yoel Capilla

THE PROBLEM

Move to a new city, **lose**
your sports community

Meetup, Sportlerei: too
broad, or stuck to one
country

Finding people = WhatsApp
groups, random Facebook
events, asking around

International students said
this was their biggest social
problem

OUR SOLUTION

A focused app that does one thing well:
matching people to activities nearby

Local-first, built around our city and
campus

Create or join an activity in seconds, chat
with the group, keep playing

Made for newcomers and international
students first

A WINDOWS DESKTOP APP WHERE YOU CAN...

Sign up, log in, set your sports and languages

Browse activities, filter by sport

Create your own (place, time, max players)

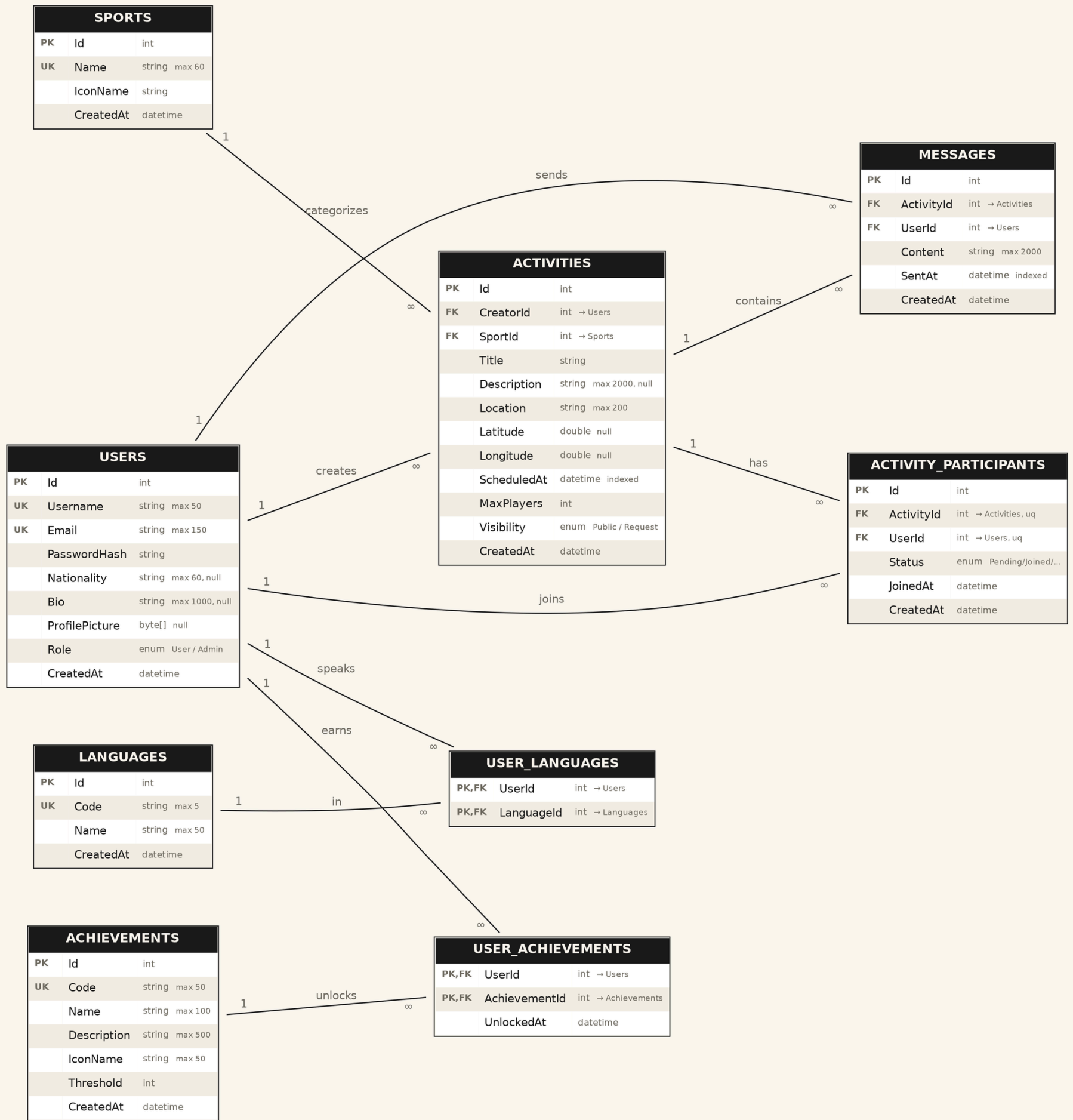
Join, leave, chat with the group

See your stats, unlock achievements

Admin tools for managing users

.NET 8 · WPF · EF Core 8 (SQLite) · MVVM · xUnit

ERD



ARCHITECTURE

1. UI → WPF + MVVM (uses Services)
2. Services → business logic (uses Data)
3. Data → EF Core (uses Domain)
4. Domain → entities + enums (no dependencies)
5. Tests → unit tests (use Services + Domain)

ARCHITECTURE & DESIGN DECISIONS

MVVM

View models know nothing about WPF windows.

Dependency Injection

Every service handed its dependencies through the constructor, easy to mock.

Repository

We use EF Core's DbSet's directly instead of wrapping them.

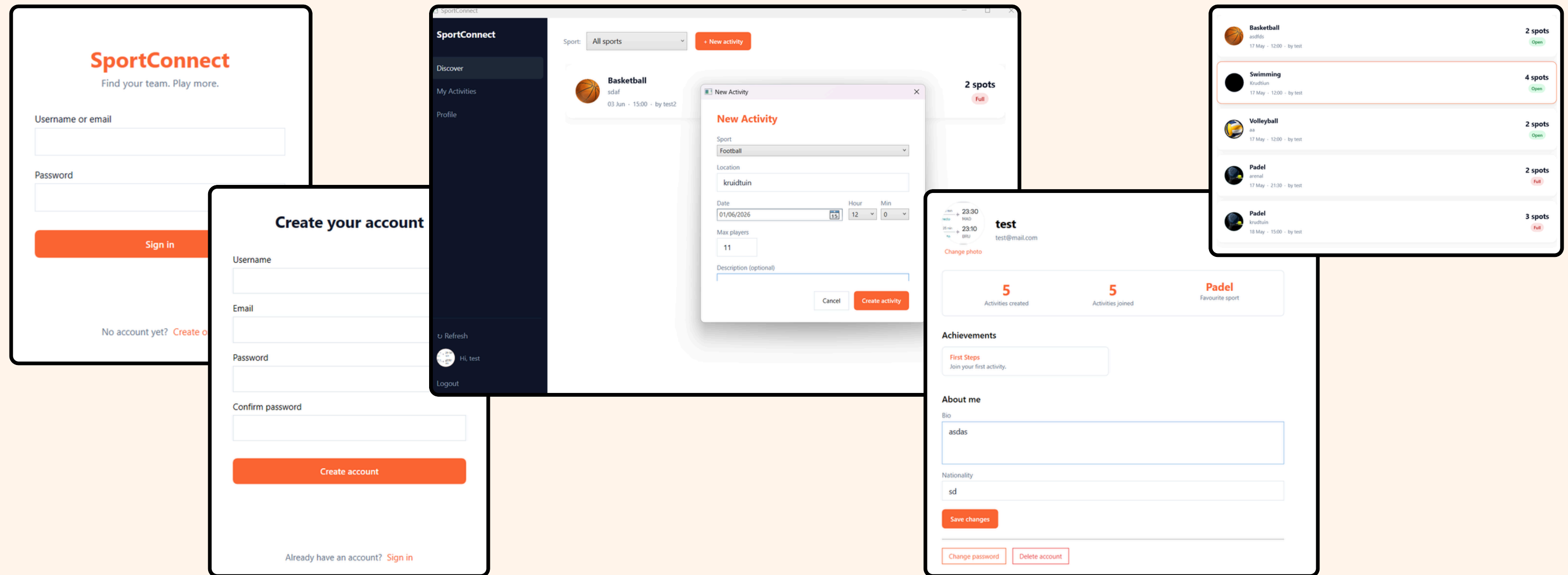
Unit of Work

Each service decides its own transaction with SaveChangesAsync.

Observer

View models raise events like LoginSucceeded and the view reacts.

OUR APP



STRATEGY PATTERN

Every achievement is a rule
behind one interface:
`IAchievementRule`

Four rules: First Steps,
Social Butterfly, Master
Creator, Dedicated Player

The service doesn't know
any rule by name, it gets
them all injected and loops
through

New achievement = one
class + one line to register
it. Nothing else changes.

TECH CHOICES

SQLite over SQL Server → single-user app, no server, ships with the app.

EF Core code-first → C# entities define the schema.

BCrypt → passwords never stored in plain text, slow on purpose to resist brute-force.

CommunityToolkit.Mvvm → source generators cut the boilerplate.

xUnit + EF InMemory → tests run in milliseconds.

TESTING

We tested the business logic, where bugs actually hurt:

Domain → IsFull,
IsChatActive

AuthService → register,
login, the bad cases

ActivityService → auto-join,
capacity, leaving

AchievementService →
rules fire, no duplicate
unlocks

StatsService → empty user,
most-played sport

Each test runs on a fresh in-memory database with a random name, so no test affects another. We tested the UI by hand, not automated.

LIVE DEMO

Register

Login, land on the feed.

Activities

Create an activity, join from another profile.

Profile

Check user's profile and its functionalities

Admin

Enter the admin account, check everything from there

WPF + MVVM

Keeping logic out of the code-behind took discipline; events from view model to view solved it.

Chat lifecycle

A chat stays open from creation until midnight of the activity day, a rule we put on the entity itself.

Achievement timing

We stopped checking on every join and now check lazily when the profile opens. Small delay, far fewer database calls.

CHALLENGES

FUTURE WORK

Future possible updates

Real-time chat with SignalR
instead of polling

A mobile app (the Domain
layer is already reusable)

A map view (we already
store coordinates)

Buddy-matching by shared
language and sport

Push notifications when
someone joins



**THANK
YOU**



**YOEL CAPILLA
ALEJANDRO VILLARREAL**